

L05: Data Exploration with Pandas and NumPy

Dr. Todd Jones

t.r.jones@reading.ac.uk

Department of Computer Science

Data Science Concepts

What is Data Science?

IBM, 2019:

- Data science is the process of using algorithms, methods, and systems to extract knowledge and insights from structured and unstructured data. It uses analytics and machine learning to help users make predictions, enhance optimization, and improve operations and decision making.

Oracle, 2020:

- Data science combines multiple fields including statistics, scientific methods, and data analysis to extract value from data.

Edureka, 2022:

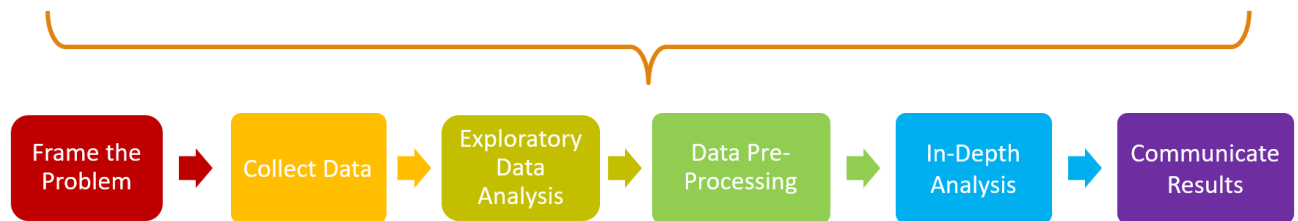
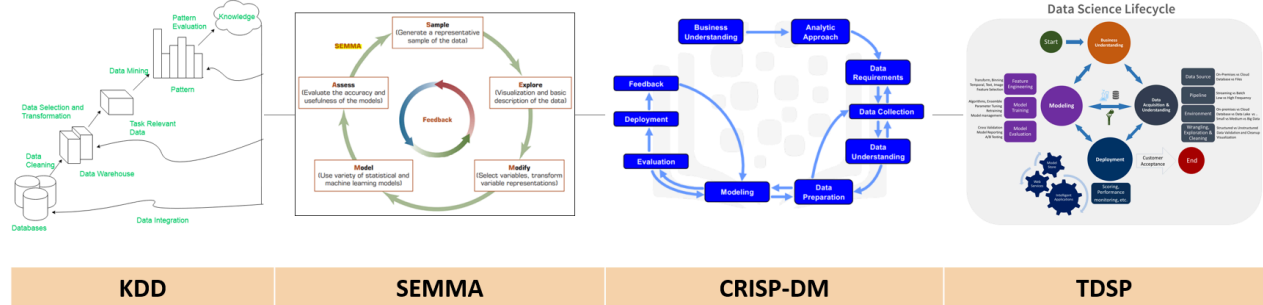
- Data Science is a blend of various tools, algorithms, and machine learning principles with the goal to discover hidden patterns from the raw data.

Key message:

While there are many possible formal definitions, from data, we need to extract knowledge and insights by implementing scientific analysis methods and machine learning algorithms.

The wealth of data being collected and stored can bring transformative benefits to organizations and societies — **but only if we can interpret it.**

Our Data Science Process



KDD: Knowledge Discovery in Databases

- <https://www.geeksforgeeks.org/kdd-process-in-data-mining>

SEMMA: Sample Explore Modify Model Assess

- <https://sis.binus.ac.id/2021/09/30/data-mining-semma>

CRISP-DM: Cross Industry Process for Data Mining

- <https://www.datascience-pm.com/crisp-dm-2>

TDSP: Team Data Science Process

- <https://learn.microsoft.com/en-us/azure/architecture/data-science-process/lifecycle>

Frame the Problem

- What is the problem we are trying to solve?
- What is the project **objective/goal**?
- How can we **measure success** in the project?
- What is the relevant data, and **where** does it exist?
- Do we have the necessary **tools** to evaluate the data?
- Develop a **plan** for the first iteration of the path to solution.

In this module: this will be done for you in the examples.

Collect Data

- How is the data **accessed**?
- Where will the data be **stored**?
- Maybe we just need a sample for **Exploratory Data Analysis**.

- Assess broad **characteristics** of the data:
 - Size
 - Type/format
 - Features
 - Quality
 - Ethical issues
- Do we need to develop a pipeline for new or updated data?

In this module: this will be done for you, or you will be provided with the means to collect the data.

Exploratory Data Analysis

EDA



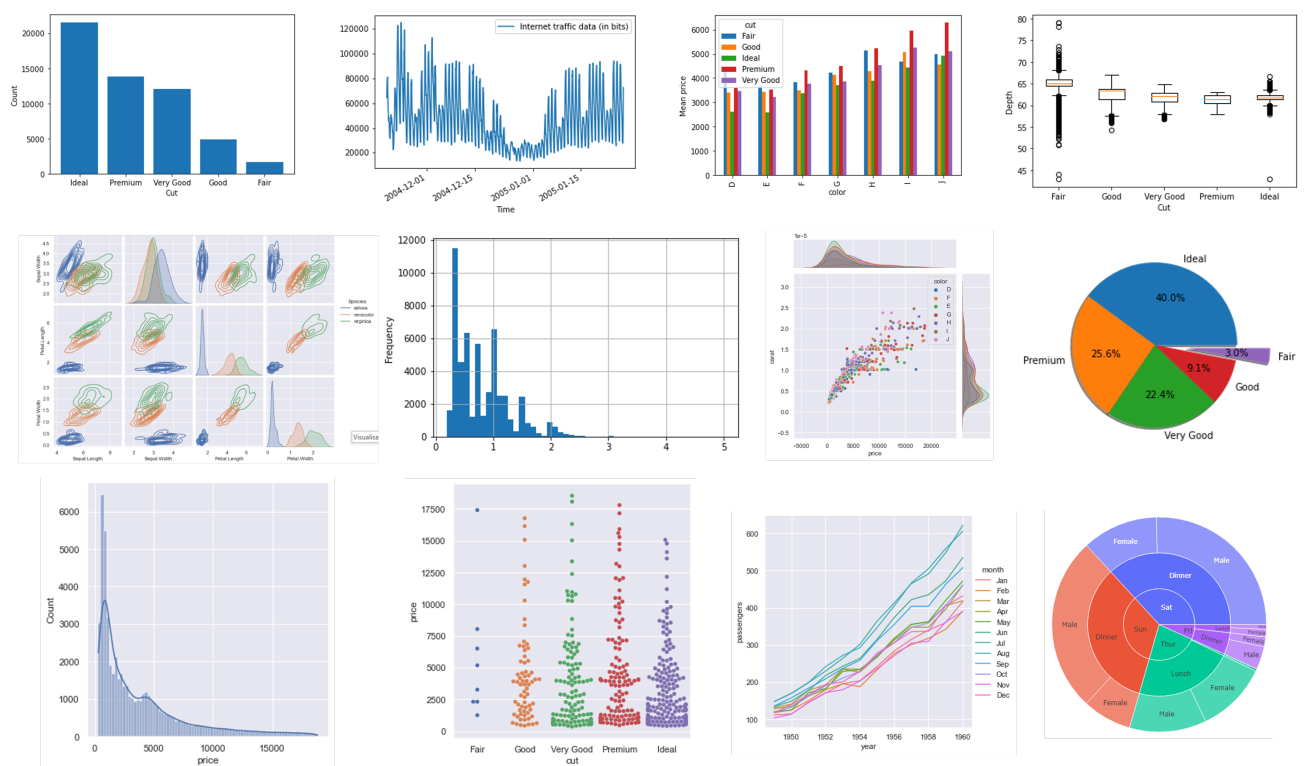
- Load the data and perform **preliminary assessments** with heavy use of data **visualisation**.
 - Fundamentally, a **creative** process.
 - **Be brave!**
- Defined and popularized by American mathematician, **John Tukey**, in the 1970s, literally writing the book on it.
 - Many processes bear his name.
 - Invented the **box plot**.

-
- Prevents us from making **assumptions** about the data.
 - Note the data's characteristics:
 - **Structure**
 - Shape, size, orientation
 - **Definitions**
 - **Descriptive statistics**

- What are the ranges and distributions of variables in the data?
 - Do you see:
 - Duplicated records?
 - Missing values?
 - Incorrect values?
 - Outliers?
 - How do the variables **correlate** with one another?
-

How does data type dictate analysis?

- **Qualitative** (e.g., strings), "categorical"
 - Not easily counted or expressed numerically
 - Country of origin
 - Gender
 - Name
 - Hair colour
 - Emotional state
 - Phase of matter
 - **Nominal**: *named*, cannot be ranked
 - **Ordinal**: *ordered*, can be ranked
 - Correspond to a level of detail (**granularity**) in a data visualisation
 - Categorise and segment data
- **Quantitative** (e.g., numbers), "numerical"
 - Countable, measurable, numerically expressible
 - Height
 - Weight
 - Age
 - Temperature
 - Price
 - Attentive students
 - **Discrete**: *integers*, separate and distinct
 - **Continuous**: *floats*, parts of an unbroken whole without interruption
 - Correspond to calculable elements
 - Aggregations: sum, min, max, mean, count, etc.



How do we classify these types of data?

- Atoms in a molecule
- Strawberries, grapes, lemons, and pineapples
- Motor race qualifying times
- Dissatisfied, slightly satisfied, neutral, very satisfied, extremely satisfied
- Currency

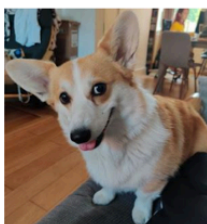
Data Pre-Processing

- Define and select the amount and kind of data for the full analysis (**scope**).
 - Assure it is **fit for consumption**, meeting the needs of your model and the data consumers.
 - Consistent and unambiguous.
- Define a procedure to address issues in data quality (**cleaning**).
 - **Noise**: some modification of the original values (e.g., distortion of a person's voice during a telephone call)
 - **Outliers**: data objects with characteristics that are considerably different from most of the other members of the set
 - **Duplicated values**: repeated entries or multiple entries (e.g., 1 person with 2 ages)
 - **Missing values**: data was not collected or was lost
 - Eliminate
 - Ignore
 - Estimate
 - Define a default
 - Assign to most common value

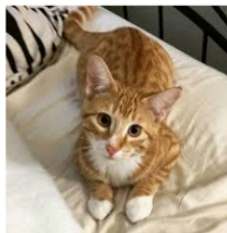
- Assign to the mean or median
 - Interpolate or extrapolate
 - Fit to a curve
- Define how the data might be **transformed** or **reduced**.
 - To make the data better fit the upcoming analyses and machine learning algorithms.
 - **Correlated** features might be removed from consideration.
 - Data might best be **transformed** to fit the requirements of a model.
 - **Normalisation/Standardisation**: transform the **scale** of the variables to reduce bias or improve processing speed
 - Min-Max (feature scaling)

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$
 - Z-score (standard score)

$$x_{scaled} = \frac{x - x_{mean}}{x_{stddev}} = \frac{x - \mu}{\sigma}$$
- Engineering of **features** to provide the most relevant variables that will enhance the quality of the results.
 - **Ordinal Encoding**: Assign an integer value to a particular label.
 - Good for naturally ordered relationships (e.g., 1st, 2nd, 3rd,...)
 - **One-Hot Encoding**: Transform labels into binary array, with 1 denoting a specific label.
 - Good for non-ordinally-related data (e.g., dog, cat, lizard)
 - **Dummy Variable Encoding**: Similar to one-hot, but representing C categories with C-1 binary variables.
 - Reduces dimensionality of data.
 - E.g., for 3 colours RGB, (R=0; B=0) implies G=1
 - Calculating new feature values
 - Calculate BMI from height and weight data



- Ordinal Encoding: 1
- One-Hot Encoding: [1,0,0]
- Dummy Variable Encoding: [1,0]



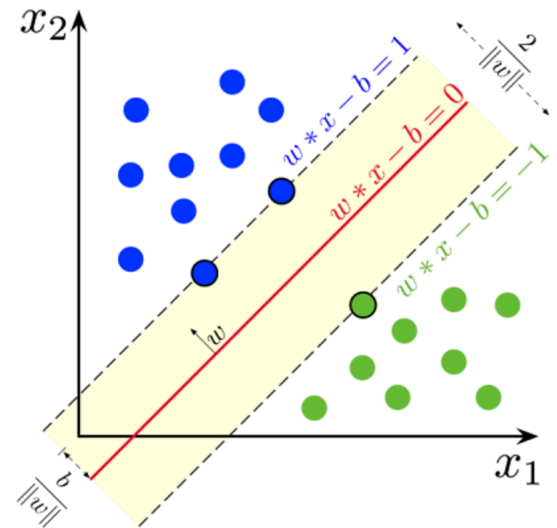
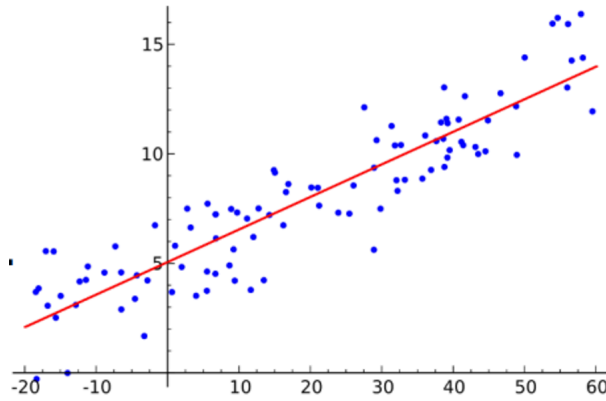
- Ordinal Encoding: 2
- One-Hot Encoding: [0,1,0]
- Dummy Variable Encoding: [0,1]



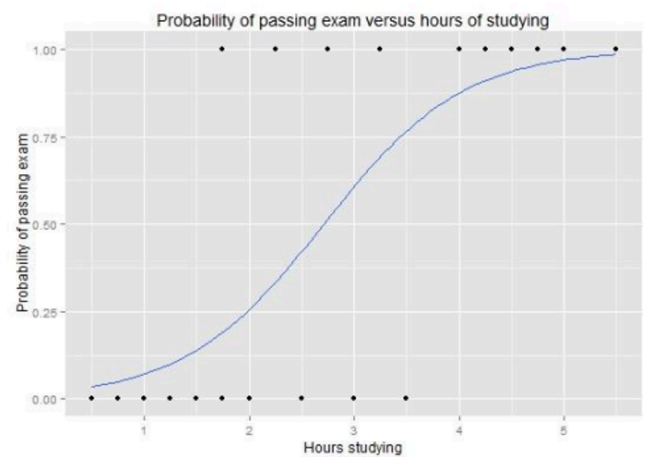
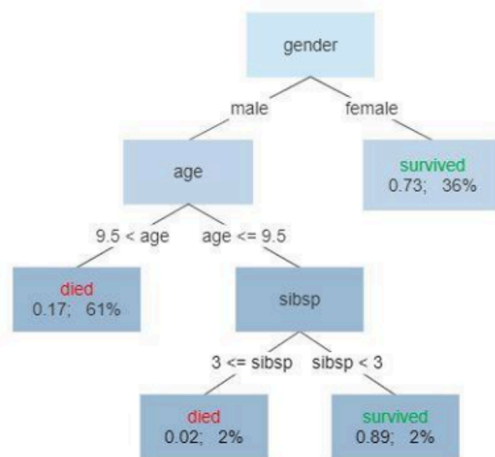
- Ordinal Encoding: 3
- One-Hot Encoding: [0,0,1]
- Dummy Variable Encoding: [0,0]

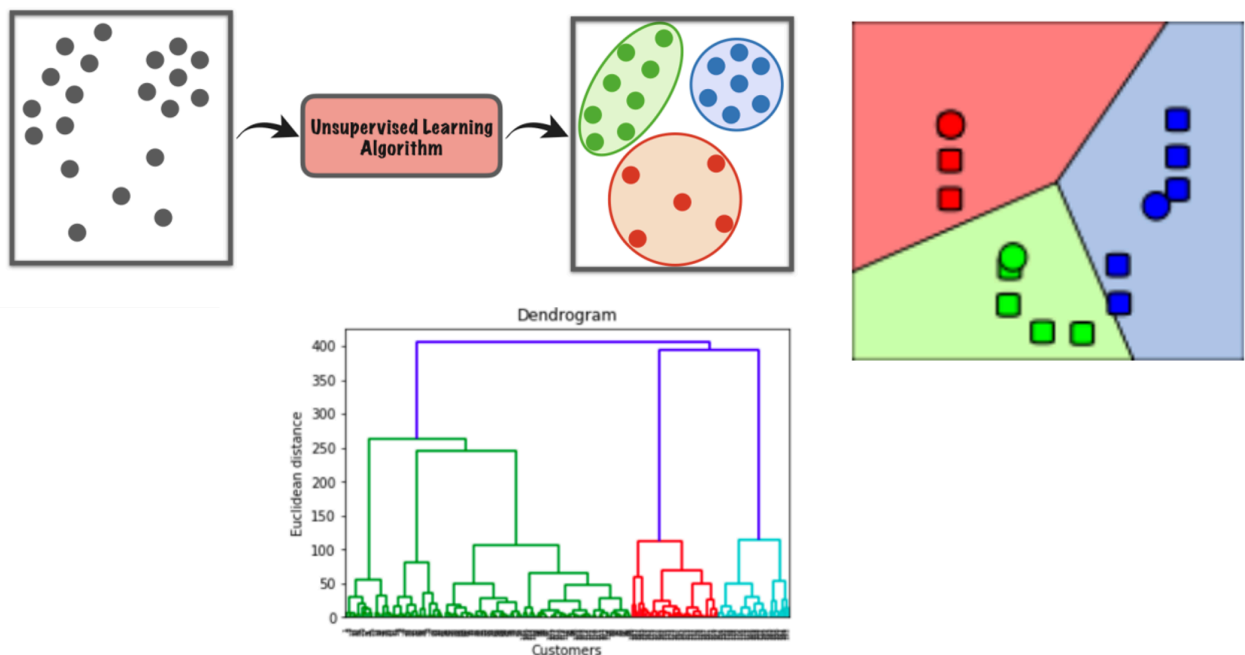
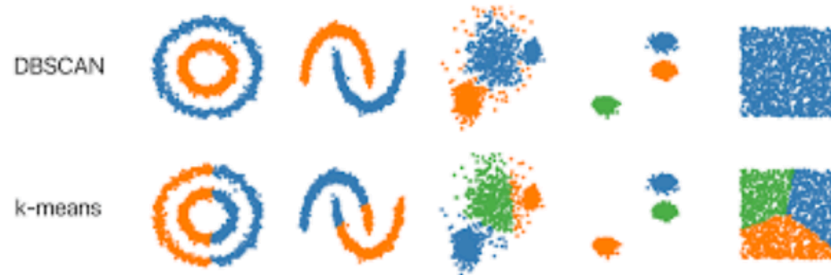
In-Depth Analysis

- **Apply** machine learning and statistical modelling techniques.
 - Supervised and unsupervised methods
 - Regression and correlation
 - Time series analysis
 - Clustering
 - Classification



Survival of passengers on the Titanic





- **Test** multiple algorithms.
- **Iteration**, retuning.
- Model **evaluation**.
 - Select appropriate **metrics**.
 - **Train/test** split.

More on this later...

Communicate Results

- Create **reports**, presentations, websites, spreadsheets, dashboards, etc.
- **Deploy** the models for use in prediction.
- **Assess** operational performance.
- **Evaluate** the need to revisit earlier steps in the process.

In this module: we assume that you have a general understanding of how to do this, and you will be asked to practice your presentation skills in the assessments.

Python Libraries for Data Analysis

A **library** is a bundle of code (perhaps multiple modules or packages) made to help you accomplish routine tasks more quickly.

Two very commonly used Python libraries for data analysis:

- **pandas**: for structured data operations and manipulations
- **NumPy**: for mathematical functions, statistics functions, scientific computing, etc.

`pd` and `np` are the **standard abbreviations** to use to refer to the elements of the libraries.



```
In [ ]: import pandas as pd
import numpy as np
```

```
In [ ]: print(f'pandas version: {pd.__version__}')
print(f'numpy version: {np.__version__}')
```

Locate your **version-specific** documentation:

- <https://pandas.pydata.org/pandas-docs/version/3.0/index.html>
- <https://numpy.org/doc/2.3/index.html>

Functionality and implementation can change **significantly** across versions.

Pandas Data Structures

- While `pandas` is named after the now-deprecated 3-D `panel` data structure, the most widely used `pandas` data structures are the `Series` and the `DataFrame`.
 - A `Series` is similar to a **single column** of data.
 - A `DataFrame` is similar to a table with **rows** and **columns**.
-

Both can have their data indexed with **labels** to refer to specific locations or subsets of their data arrays.

Series

`Series` in `pandas` are **one dimensional arrays** where each index in the array is labelled. They are constructed using:

`pandas.Series(data, index)`

- The `data` argument must be either an array or list, a dictionary, or a scalar value (`int` or `float`).
- If no `index` is given, then one is constructed using integers:
 - `0, 1, ..., len(data)-1` .

```
In [ ]: # Basic, unlabelled Series creation:
sr1 = pd.Series([3, 4, 5])
sr1
```

```
In [ ]: # Basic, labelled Series creation:
sr2 = pd.Series([3, 4, 5], index=['one', 'two', 'three'])
sr2
```

```
In [ ]: # Labelled Series from dictionary data:
dc = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
sr = pd.Series(dc)
sr
```

Indexing Series

`Series` in `pandas` are **ordered**.

Values can be accessed both by their `index label` and also by a numeric index.

numeric index	index	value
0	a	1
1	b	2
2	c	3
3	d	4

```
In [ ]: # Individual index
print(sr[-1]) ## could confuse the system if int indices present or mixed
print(sr['d'])
```

```
In [ ]: # Slicing
print(sr[1:2])
print(sr['b':'c']) # End index is INCLUSIVE!
```

```
In [ ]: # Specify list of index labels
print(sr[['a', 'd']])

# Note type representation
type(sr[['a', 'd']])
```

DataFrame

- Data can be stored in **tables** with one column for each **attribute/feature/variable**, and one row for each data **object/item**.
- A `DataFrame` is a tabular data structure for storing data that have **multiple attributes**.
 - Each **column** can have a different data type.
- Just as with a `Series`, every **row** in a `DataFrame` has an index.
- `DataFrame`s can be constructed using:

`pandas.DataFrame(data, index, columns)`

If your `data` argument comes from a Python `list`, then each entry of the `list` will correspond to a `DataFrame` **row** index.

- `index`: row label
- `column`: column label

```
In [ ]: # Basic manual DataFrame creation, automatic index values
df1 = pd.DataFrame(data=[[1, 2, 'r'],
                        [3, 4, 's'],
                        [5, 6, 't'],
                        [7, 8, 'u'],
                        [9, 10, 'v']],
                  columns=['column_1', 'column_2', 'column_3'])
df1
```

```
In [ ]: # DataFrame creation from a dictionary of lists:
d = {'dict_column_1': [1, 3, 5, 7, 9],
     'dict_column_2': [2, 4, 6, 8, 10],
     'dict_column_3': ['r', 's', 't', 'u', 'v']}
df2 = pd.DataFrame(d)
df2
```

```
In [ ]: # Inspect data types
df2.dtypes
```

Indexing `DataFrame`s

This is straightforward for individual columns:

```
In [ ]: # Specify individual column name
df2['dict_column_2']
```

```
In [ ]: # Use object notation
df2.dict_column_2
```

This is straightforward for lists of columns:

```
In [ ]: df2[['dict_column_1', 'dict_column_3']]
```

But we cannot **slice** columns in this way:

```
In [ ]: df2[:, 'dict_column_2']
```

And if a single argument corresponds to a column, how can we index a row?

`.loc[]` and `.iloc[]`

`DataFrame` s have special `loc` and `iloc` attributes that allow for indexing.

These permit **slicing**:

```
In [ ]: # Display again for reference  
df2
```

```
In [ ]: # Index using index and column references  
# [rows, columns]  
  
# ALL columns  
df2.loc[1:3]
```

```
In [ ]: # ALL rows of one column (a Series)  
  
print(type(df2.loc[:, 'dict_column_2']))  
  
df2.loc[:, 'dict_column_2']
```

```
In [ ]: # A subset, sliced in both directions  
# End reference INCLUSIVE  
df2.loc[1:3, 'dict_column_2':'dict_column_3']
```

```
In [ ]: # In contrast, use iloc  
# End reference EXCLUSIVE  
df2.iloc[1:3, 1:2]
```

Constructing DataFrames from Series

We have seen that an isolated column from a `DataFrame` has `Series` type.

It follows that `DataFrame` s can be built by grouping together `Series` , with each `Series` making up a column of the `DataFrame` .

The indices of the `Series` will be used to construct the indices of the rows of the `DataFrame` .

```
In [ ]: # Construct DataFrame from multiple Series objects  
# using random numbers as data  
a = pd.Series(np.random.randn(4),  
              index=['one', 'two', 'three', 'four'])  
b = pd.Series(np.random.randn(4),  
              index=['one', 'two', 'three', 'four'])  
c = pd.Series(np.random.randn(4),  
              index=['one', 'two', 'three', 'four'])  
  
# Construct the DataFrame from a dictionary of Series objects
```

```
dfs = pd.DataFrame({"A": a, "B": b, "C": c})
dfs
```

```
In [ ]: # Or a bit more Pythonically, using the Series name attribute:

a = pd.Series(np.random.randn(4),
               index=['one', 'two', 'three', 'four'], name='A')
b = pd.Series(np.random.randn(4),
               index=['one', 'two', 'three', 'four'], name='B')
c = pd.Series(np.random.randn(4),
               index=['one', 'two', 'three', 'four'], name='C')

pd.DataFrame({x.name:x for x in [a, b, c]})
```

Appending Entries to DataFrames

When compiling and updating `DataFrames`, it is useful to add individual or multiple entries to an exiting structure.

To do this, use the `.append()` method (now deprecated) or `.concat()`.

- **Along a shared index or column**
- Combining separate files
- Making multiple API requests
- <https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.concat.html>

```
In [ ]: # Append a DataFrame with same column names
# Returns DataFrame, need to reassign variable to store result

# Deprecated; no longer available!

## dfs.append(dfs.iloc[0])
```

```
In [ ]: # Concatenate two DataFrames
# Slicing in this way returns a DataFrame
# instead of a Series --> dfs.iloc[1]

# Returns DataFrame, need to reassign variable to store result

pd.concat([dfs, dfs.iloc[1:2]])
```

Here, you are actually trying to concatenate a `DataFrame` with a `Series` in the latter two cases.

Pandas is happy enough to let you do this, but the logic it uses is not intuitive.

When you concatenate a `Series` to a `DataFrame`, `pandas` treats the `Series` as a *column*, not as a row.

- It tacks the Series on as a new column.
- It aligns on the index (which likely does not match anything), so it fills with NaNs.

Use **double brackets** on `.iloc[[]]` or `.loc[[]]` or give a `.loc[same:same]` range to indicate that you want a `DataFrame` back.

- The double brackets ensure you get a `DataFrame` with one row, not a `Series`.
- `pandas` interprets `.loc[same:same]` as a *slice*, not a point lookup.

`DataFrames` can also be concatenated by adjacent **columns**.

In []:

Further, `pd.merge()` and `df.join()` can be used to merge objects with a database-style join operations:

- `how`: left, right, outer, inner, cross
- <https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.merge.html>
- <https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.join.html>

Please investigate these methods on your own.

Warning!

Beware that objects returned by indexing a subset of a `Series` or `DataFrame` object in `pandas` are (generally) **views** of the original object, not **copies**, and modifications will change the corresponding elements in the original `Series` or `DataFrame` object.

Avoid this by using `.copy()` to make a copy of the `DataFrame` as necessary.

Indices in pandas

The row and column names of `pandas` objects are stored in `Index` or `RangeIndex` objects.

You can access the columns and indices (rows) of a `DataFrame` or the indices of a `Series` using the `columns` and `index` attributes.

```
In [ ]: # Inspect the DataFrame indices
df2.index
```

```
In [ ]: # Inspect the DataFrame columns
df2.columns
```

Reading Data with pandas

It would be an absurdly laborious task to ask each of you to create large datasets manually.

Pleasantly, interesting data already exists in some structured format, and we simply need to know how to **read** it.

HTML

`pandas` can scrape tables from HTML, given a URL string. A Python `list` of all tables scraped from the web page will be returned.

https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.read_html.html

```
In [ ]: # Mathematical, Physical and Computational Sciences - Modules for 2026-2027

url = 'https://www.reading.ac.uk/modules/module?acyear=2026%2f7&school=MPS%20CS|MPS%20MATHST|I

modules = pd.read_html(url)
print(type(modules))
print(len(modules))
```

```
In [ ]: # Inspect the scraped table
modules[0]
```

Displaying all rows

```
In [ ]: pd.set_option('display.max_rows', None)
modules[0]

## Output removed for alternate versions, too long.
```

```
In [ ]: # Reset to default display
pd.set_option('display.max_rows', 10)
modules[0]
```

JSON

JSON (JavaScript **O**bject **N**otation) is a way of representing data structures as text.

The syntax is very similar to dictionaries and lists in Python. `pandas` can construct a `DataFrame` from a JSON string or file.

```
In [ ]: import pandas as pd
from io import StringIO # Required for json_read, which expects "file-like" input
```

```
In [ ]: # Create DataFrame from JSON string
json_string = '{"A":[1,2,3], "B":[4,5,6], "C":[7,8,9]}'
j = pd.read_json(StringIO(json_string))
j
```

Addressing the `FutureWarning`

In earlier versions of `pandas`, `read_json` could accept string literals. Eventually, the ambiguity of whether it would be a string literal or a file name became awkward. The developers implemented a `FutureWarning` to be displayed where string literals were passed:

```
# FutureWarning: Passing literal json to 'read_json' is deprecated and will be
removed in a future version. To read from a literal string, wrap it in a 'StringIO'
object.
j = pd.read_json(StringIO(json_string))
```

Implemented as a warning, users could continue to successfully pass string literals, while being informed that a code change would prevent them from doing so in a future `pandas` version release.

```
In [ ]: from io import StringIO

json_string = '{"A":[1,2,3], "B":[4,5,6], "C":[7,8,9]}'

'''
StringIO is a class that creates an in-memory file-like
object that behaves like an open text file.
Future implementations will expect to open a file with the
first argument, and we are making the string "like a file".
'''

# Wrap the string in StringIO so pandas reads it as a buffer.
j = pd.read_json(StringIO(json_string))

print(j)
```

Example Dataset: Chicago Bike Share

Live public JSON feed of data from bike stations in Chicago.



- View the data structure in your browser:
https://gbfs.divvybikes.com/gbfs/en/station_information.json
- A dataset containing a list of all stations with a value for each station attribute.

This data and more available from <https://www.divvybikes.com/system-data>.

```
In [ ]: # requests HTTP Library
# https://requests.readthedocs.io/en/latest/
import requests

bikes_url = "https://gbfs.divvybikes.com/gbfs/en/station_information.json"

# Return an HTML request code
bikes_data = requests.get(bikes_url)
bikes_data
```

```
In [ ]: # Transform the returned data to json format
# The json data structure has a dictionary with a 'data' key,
# containing all the data.
# Within this the 'stations' key is a list of dictionaries containing
# info about each bike station
bikes_stations = bikes_data.json()['data']['stations']
# comment for file bikes_stations

In [ ]: # Normalize into a flat table
# https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.json_normalize.html
bikes = pd.json_normalize(bikes_stations)
bikes

In [ ]: # Inspect column names
bikes.columns
```

CSV Files

Tabular data is often stored in text files, with one line per row of the table, and special characters separating the columns.

- See `./data/diamonds.csv` in this notebook archive for an example of a **comma-separated values** file.

Reading the contents of a CSV to a `DataFrame` is easy with `pandas`.

https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.read_csv.html

There are many optional parameters associated with `read_csv`. For instance, you can specify the separator, `sep`, used to format the data.

- `read_csv` will default to a **comma** `,`,
- Similarly, `read_table` will default to a **tab** `\t`.
 - https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.read_table.html

```
In [ ]: # Read in the diamonds dataset
diamonds = pd.read_csv('data/diamonds.csv')

# Display the first 5 rows of the DataFrame
diamonds.head()
```

```
In [ ]: # Display the last 5 rows of the DataFrame
diamonds.tail()
```

-
- Where are the **qualitative** data?
 - Nominal or ordinal?
 - Where are the **quantitative** data?
 - Discrete or continuous?
-

Context is critical!

Column	Description
price	price in US dollars (\$326-\$18,823)
carat	weight of the diamond (0.2--5.01)
cut	quality of the cut (Fair, Good, Very Good, Premium, Ideal)
color	diamond colour, from J (worst) to D (best)
clarity	a measurement of how clear the diamond is (I1 (worst), SI2, SI1, VS2, VS1, VVS2, VVS1, IF (best))
x	length in mm (0--10.74)
y	width in mm (0--58.9)
z	depth in mm (0--31.8)
depth	total depth percentage = $z / \text{mean}(x, y) = 2 * z / (x + y)$ (43--79)
table	width of top of diamond relative to widest point (43--95)

Exploring Our Data

What are we working with?

info()

- Prints a concise summary of a `DataFrame`'s contents.
 - Number of entries
 - How they are indexed
 - Descriptions of column features
 - Index number
 - Column name
 - Count of non-null values
 - Data types
 - Summary of data types
 - Memory usage

<https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.info.html>

```
In [ ]: diamonds.info()
```

describe()

- This gives us various statistics of the data:
 - **count** - total number of values in the column
 - **mean** - average of values in the column
 - **std** - standard deviation of values in the column
 - **min** - minimum value in the column
 - **25%** - 1st quartile, 25% of the values are less than this value
 - **50%** - median, 50% of the values are less than this value

- **75%** - 3rd quartile, 75% of the values are less than this value
- **max** - maximum value in the column
- Any **missing** values are ignored in the above calculations.
- Note that these statistics only make sense **for numerical variables**, so the columns with other forms of data (i.e., **qualitative**) are omitted.
- Also note that not all numerical values can be meaningfully analysed in this way (e.g., the mean of a numerical ID is not meaningful).

<https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.describe.html>

```
In [ ]: diamonds.describe()
```

Summarising Data

We can **operate** on the data to obtain various measures directly.

`DataFrames` have aggregate operator methods that return metrics **for specified features**, for example:

- `.sum()`
- `.min()`
- `.max()`
- `.mean()`

```
In [ ]: # Display the mean values of carat and price
diamonds[['carat', 'price']].mean()
```

`.unique()`

- List the **unique** values in a **single Series** .
 - Good for `str` categorical data.

```
In [ ]: # Display the unique strings for the color attribute
diamonds.color.unique()
```

`.value_counts()`

- Count how many times each unique value occurs in a **single Series** .

```
In [ ]: diamonds.clarity.value_counts()
```

`pd.crosstab()`

- A **function** of `pandas` , as opposed to a method of a `DataFrame` , this computes a simple cross-tabulation of two or more features of the data.
 - Good for grouping by `str` categorical data.

```
In [ ]: # Cross-tabulation between cut and clarity features

pd.crosstab(diamonds['cut'], diamonds['clarity'])
```

Missing data

`pandas` represents missing numeric values using **NaN** (Not a Number, `nan`) values.

There are several methods to detect and drop missing values from a `DataFrame`.

- `isnull()` / `isna()`
 - generates a data structure of boolean values, indicating whether each value is missing
 - <https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.isnull.html>
- `dropna()`
 - drops some rows (default) or columns (`axis=1`) if they contain missing data
 - <https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.dropna.html>
- `fillna()`
 - fills in missing values
 - <https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.fillna.html>

```
In [ ]: # Create missing boolean values
diamonds.isnull().head()
```

```
In [ ]: # Count the number of missing values
# in each column
# sum is performing a mathematical operation!
diamonds.isnull().sum()
```

```
In [ ]: # Drop rows with missing values
# Does NOT modify original DataFrame
# without inplace=True
diamonds.dropna()
```

```
In [ ]: # Demonstrate that original data unaffected
diamonds.head()
```

```
In [ ]: # Fill missing data with specified values
diamonds.fillna('MISSING').head()
```

Identifying and Dropping Irrelevant Data

Some methods for detecting and dropping irrelevant data:

- `uplicated()`
 - returns a boolean `Series` denoting duplicated rows.
- `drop_duplicates()`

- drops the duplicated rows (keeping the first occurrence by default).

- `drop()`
 - drops specified labelled rows (default).
 - can be used to drop irrelevant columns, `columns=[...]`.

We will examine these methods in **Practical 5**.

Creating New Columns

We often would like to augment our datasets with additional data.

- If we have an array of data matching the length of the DataFrame, we can add a column in a manner similar to adding an entry to an existing dictionary:

```
In [ ]: # Check Length
df_len = len(diamonds)

# Create data of same length
aug_data = ['aug'] * df_len

# Add column to DataFrame
# Original DataFrame is updated.
diamonds['Augmented'] = aug_data
diamonds.head()
```

Vectorisation

Through the process of **vectorisation**, we *broadcast* an operation over a vector or array of data.

- Vectorised operations are **element-wise**.
 - This differs from *matrix*-style operations.
- In `pandas`, you can expect all mathematical operations to be applied *element-wise*.

Applying Functions

There are several ways in which available **functions** can be applied to `DataFrame` contents.

One of these is the `df.apply()` method.

- Applies a function **along an axis** of the `DataFrame`.
- <https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.apply.html>

With this, we can apply any number of custom functions to our data.

```
In [ ]: # This is an example of feature engineering: discretisation/binning
# See also: pd.cut()
# Discretisation can improve interpretability, effects of nonlinearity, variability (noise)
# or be helpful to represent domain class definitions
# May lead to information loss, problem with bin edge definitions/sensitivities, discontinuities
# It is the data science equivalent of rounding off decimals and acting surprised when precise
```

```
def assign_table_category(x):

    if x >= 80:
        return 'large'
    elif x >= 60:
        return 'medium'
    else:
        return 'small'

# Apply function to table data, store result
diamonds['table_category'] = diamonds.table.apply(assign_table_category)

# Display
diamonds.head()
```

Conditionally Selecting Data [boolean indexing]

- Boolean indexing: `DataFrame[condition]` returns the `DataFrame` consisting of rows that satisfy the *condition*.
- **Create** with comparison operators (`==` , `!=` , `>` , `>=` , `<` , `<=`) and logical operators (`and` , `or` , `not`) are used to write the *condition*.
- Conditions can be **combined** with bit operators: `&` , `|` , and `~` , which can be broadcast across an array of values.

Why?

- **Filtering data** for analysis, reporting, or visualisation.
- **Creating subsets** for modelling, cross-validation, or exploratory analysis.
- **Constructing masks** for further computation (e.g., replacing, imputing).
- **Debugging** ("what's going on in this subset?").

```
In [ ]: # Display diamonds with Ideal cut only
diamonds[diamonds['cut'] == 'Ideal'].head()
```

```
In [ ]: diamonds['cut'] == 'Ideal'
```

```
In [ ]: # Display diamonds with only Ideal cut and color E

# NOTE: use of brackets...order of operations!

ideal_E = diamonds[(diamonds['cut'] == 'Ideal') & (diamonds['color'] == 'E')]
ideal_E.head()
```

```
In [ ]: # Display the average price of diamonds with ideal cut and E color
# and table less than 60
ideal_E[ideal_E.table < 60].price.mean()
```

See also `DataFrame` methods:

- `.isin()`
- `.str.contains()`

- `.between()`
- `.where()`
- Negations: `df[~df['valid']]`

NOTE: Comparisons involving `NaN` always return `False`.

- Filtering like `df[df['col'] > 5]` silently drops `NaN`s unless you are careful and address handling of the missing values specifically.

Wide and Long Data

So far, we have been working with mainly **wide** data, organised with:

- columns corresponding to **variables or attributes**
- rows corresponding to the **objects** we are collecting data for
- **values** at each data point
 - Any number of these might be **missing**

Index	Height	Length	Width

Another way of organising the same data is to store it as **long** data, where:

- each row stores entries for the **variable name** and **value** columns, and optionally a corresponding **key** column.

Long data can be useful where:

- not all variable values are recorded for all objects
- new/different variables are recorded for different objects (the method of recording might have changed over time).
 - This means that **no missing** values are typically stored in this format, which can be **more efficient**.

Index

Variable

Value

	Height	
	Length	
	Width	
	Height	
	Length	
	Width	

Another reason that the difference matters is because **certain tools and methods expect one format or the other**.

We can use `melt()` on our **wide** data to give it a **long** format.

Conversely, long-format data can be converted to wide data with `pivot()`.

Changing the format can make certain operations simpler and provide proper formatting for certain kinds of plots.

```
In [ ]: # Re-display for reference
diamonds.head()
```

```
In [ ]: # This works best when each entry has a unique identifier
diamonds['ID'] = diamonds.index

# ALL of the original column names become
# entries under the "variable" column.
```

```
# Data values are moved to the "data" column.
```

```
# ID data is stored in its own column
```

```
long = diamonds.melt(id_vars='ID')  
long
```

```
In [ ]: # Convert the Long data back to wide format  
long.pivot(index='ID', columns='variable', values='value')  
  
# NOTE: Note that pivot() assumes one unique value per combination.  
#       If there is duplication, it will raise an error.
```

groupby()

The `groupby()` method groups rows together based on the **values in a specific column**.

Once grouped, we can then **apply functions to summarise the values in each group**.

This is a shortcut for filtering the `DataFrame` for rows associated with each unique entry for a feature, then calculating a summary statistic for each group, one at a time.

- **Typically, we group by qualitative categories.**

Imagine that the **wide** data table was a plant dataset with a "species" attribute for each object.

- We can group by the unique species categories to quantify species-specific measurements.



```
In [ ]: # Group the diamonds by their qualitative 'cut' attribute  
diamonds.groupby('cut') # this returns a DataFrameGroupBy object
```

```
In [ ]: # Display the statistics of the grouped values  
diamonds.groupby('cut').describe()
```

```
In [ ]: # Display summary statistics for selected columns  
diamonds.groupby('cut')[['price', 'x', 'y', 'z']].mean()
```

The `.agg()` method can produce multiple statistics at once.

- Over a specified axis; defaults to 0:
 - consider all rows for each column.
- Operates on provided **functions**.

<https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.agg.html>

```
In [ ]: # Compute the min, mean, and max price for each cut category
diamonds.groupby(['cut']).agg({'price': ['min', 'mean', 'max']})
```

```
In [ ]: # Form groups based on multiple features
diamonds.groupby(['cut', 'clarity'])[['price', 'carat']].mean()
```

The `.filter()` function will apply a function that returns `True` or `False` to each group grouped data.

For instance, we could try to **reduce** the size of our dataset so that it only includes clarity categories with more than a certain number of entries.

<https://pandas.pydata.org/docs/reference/api/pandas.api.typing.DataFrameGroupBy.filter.html>

```
In [ ]: # Retain only those clarity groups that have more than 5000 entries
reduced = diamonds.groupby('clarity').filter(lambda x: len(x) > 5000)
reduced
```

```
In [ ]: # Display counts to verify
diamonds.clarity.value_counts()
```

```
In [ ]: # Display counts to verify
reduced.clarity.value_counts()
```

Writing DataFrames to File

Once you have modified your `DataFrame`, likely having created additional fields, you might like to save the result for later use or to share.

There are a variety of `DataFrame.to_` methods.

To write to a CSV file, use:

```
df.to_csv("filename.csv")
```

A number of optional parameters are additionally available for each of these writing methods.

- E.g., do not save the index value in the file: `index=False`

https://pandas.pydata.org/pandas-docs/version/3.0/reference/api/pandas.DataFrame.to_csv.html

Introduction to NumPy

`numpy` is a Python package designed for working with **many-dimensional** arrays *efficiently*.

"Numerical Python"

```
import numpy as np
```

```
In [ ]: import numpy as np  
np.__version__
```

NumPy is great for...

- Array construction, indexing, and slicing
- Performing efficient **vectorised** operations on arrays
- Reshaping and stacking arrays
- Linear algebra, FFT, and matrix operations
- Random number generation
- Numerical constants
 - `np.nan` , `np.inf` , `np.pi` , `np.e` , etc.
- Fast sorting
 - Much faster than Python's `sort` and `sorted` built-ins
 - Quicksort (default), mergesort, heapsort
- Integration with C, C++, and Fortran code to Python
 - Permits low-level passing of data arrays and use of very efficient legacy processing libraries
 - Enabled by contiguous, uniformly typed structures

As such, `numpy` is a **critical component** of many popular Python utilities:

- **pandas** :
 - `numpy` provides the underlying structure to `DataFrames` and `Series`
 - uses `numpy` broadcasting rules
 - data manipulation methods (`groupby` , `pivot` , `melt` , `concat` , etc.) are built on `numpy` 's reshaping functions
 - uses `numpy` routines for arithmetic, statistics, and linear algebra operations
- **matplotlib**
 - for data visualization and plotting
- **statsmodels**
 - for statistical modeling and hypothesis testing.
- **scipy**
 - for scientific computing and technical computing
- **sklearn**
 - for machine learning and data analysis
- **tensorflow**
 - for deep learning and artificial intelligence
- **torch** (PyTorch)

- for deep learning and artificial intelligence
 - `cv2` (OpenCV)
 - for computer vision and image processing
 - `astropy`
 - for astronomy and astrophysics
-

Arrays in NumPy

`ndarray` : a NumPy N-dimensional array

- Arrays have a single type (generally...see `object` types and structured record arrays to mix types).
- A **one-dimensional** array is a **vector**
- A **two-dimensional** array is a **matrix**
 - Orientation is **row-major** for 2D arrays: [row_indices, column_indices]
 - In memory, like C, a 2D matrix is stored one row at a time.
 - The **last index** changes most rapidly as we traverse the array in memory.
 - Alternatively, dimensions are **axes**: [axis=0, axis=1]
- An **N-dimensional** array has N indices, 3-D or higher
 - A **tensor** array

See: https://numpy.org/doc/stable/user/absolute_beginners.html

Creating Arrays

`numpy` arrays are objects of the `numpy.array` class, which is used to create arrays of multiple dimensions.

- Arrays can be created from **lists or tuples** (not sets)
 - Technically, elements can store Python objects like sets or strings, but this is not typical usage.
 - A **single list** will construct a one-dimensional array.
 - A **list of lists** will create a two-dimensional array.
-

Arrays can be created from lists with `np.array()` :

```
In [ ]: list0 = [0, 1, 2, 3]
list1 = [4, 5, 6, 7]
list2 = [8, 9, 0, 1]

array1 = np.array(list0)
array1
```

```
In [ ]: array2 = np.array([list0, list1, list2])
array2
```

When printing an N-dimensional array, `numpy` will print a formatted string that *looks like* a **nested list**

- Each leading bracketed group corresponds to successive slices along axis 0, with inner brackets reflecting deeper (right-most) axes.

```
In [ ]: np.array([array2, array2])
```

Inspecting Arrays

`ndarray.ndim`

- Returns the number of axes, or dimensions, of the array

`ndarray.size`

- Returns the total number of elements of the array

`ndarray.shape`

- Returns a **tuple of integers** describing the number of elements stored along each dimension of the array

`ndarray.dtype`

- Returns the name of the data type of the values stored in the array

```
In [ ]: array2.ndim
```

```
In [ ]: array2.size
```

```
In [ ]: array2.shape
```

```
In [ ]: nrows, ncols = array2.shape
print('nrows:', nrows)
print('ncols:', ncols)
```

```
In [ ]: array2.dtype
```

Slicing and Updating Arrays

Much like indexing lists using square brackets `[]`.

- `[rows, columns, ...]`
- `[axis=0, axis=1, axis=2, ...]`
- Entirely mutable, but the way this works is somewhat complex.

```
In [ ]: # Each list in array2 is a row (axis=0)
array2[1]
```

```
In [ ]: # Alternatively:  
array2[1, :]
```

```
In [ ]: # Obtain a single column:  
array2[:, -1]
```

```
In [ ]: # Obtain a single element:  
array2[2, 0]
```

```
In [ ]: # Replace a single value  
array2[1, 1] = 5000  
array2
```

```
In [ ]: # Replace a subset of values  
array2[1:, 2:] = -999  
array2  
  
# The LHS slice creates a view.  
# Assignment is in-place via broadcast
```

```
In [ ]: # Store a slice of the array (as a view)  
# Pointer and stride pattern into the  
# same memory buffer location as `base`  
subarr = array2[1:, 2:]  
subarr
```

```
In [ ]: #  
subarr.base
```

```
In [ ]: # Update **individual values**, in-place  
array2[ 0, 0] += 5.5  
subarr[-1, -2] += 5.5  
subarr
```

Data Generators

`np.zeros()`

- Return new array of given shape, populated with zeros

`np.ones()`

- Return new array of given shape, populated with ones

`np.empty()`

- Return new array of given shape, with random values (depending on the state of memory)

`np.arange()`

- Return new array populated with a specified sequence of values
- Similar to `range()` but actually exists in memory

`np.linspace()`

- Return new array with values spaced linearly

`np.random`

- `rand()` - random values from uniform distribution [0,1]
- `randn()` - random values from a standard normal distribution centered on zero
- `randint()` - random integers in a specified range
- `choice()` - selects a random element of a given collection

```
In [ ]: # zeros (pass tuple shape, specify type)
np.zeros((3, 3, 3))#, dtype=np.int64)
```

```
In [ ]: # ones (default to float)
np.ones((2, 15))
```

```
In [ ]: # empty
np.empty((6, 6))
```

```
In [ ]: # Range of integers (start, stop)
np.arange(15, 30)
```

```
In [ ]: # Range of floats, interval (start, stop, step)
np.arange(100, 200, 15.)
```

```
In [ ]: # Linear spacing (min, max, size)
np.linspace(-500, 500, 11, dtype=int).astype(np.int32)
# astype returns new array
# Be careful with:
# float--> int
# complex-->real
# nans
```

```
In [ ]: # Random uniform [0,1] values (*dimension_sizes)
np.random.rand(10)
```

```
In [ ]: # Random uniform [0,1] values (*dimension_sizes), multi
np.random.rand(3, 3, 3)
```

```
In [ ]: # Random normal values (*dimension_sizes)
np.random.randn(100)
```

```
In [ ]: # Random integers (start, stop, shape)
np.random.randint(1, 1000, (2, 2, 2, 2))
```

```
In [ ]: # Choose a random value from a given collection
np.random.choice([5, 55, 555, 5555, 55555, 555555])
```

Aside: Another indexing technique: `...`

- Can only be used one time in an index specification.

```
In [ ]: # 6-dimensional array
# e.g.: [case_id, variable_id, x, y, z, time]
d6 = np.random.randn(3, 3, 3, 3, 3, 3)
d6

## Output removed for alternate versions, too Long.
```

```
In [ ]: # Indexing with ellipses
# Specifically elements 2 from the first two dimensions
# and the last element of the last dimension
```

```
# then all of dimensions 3-5
# Returns a 3-dimensional array
d6[2, 2, ..., -1] # [2, 2, :, :, :, -1]
```

Reshaping Arrays

- Any `ndarray` can be **reshaped** to an array of different dimensionality **and the same number of elements** with `ndarray.reshape()`.
 - If possible, the result is a **view** of the original, not a copy.
- A multi-dimensional array can be **collapsed** into one dimension with `ndarray.flatten()`.
 - Returns a **copy**. A view is returned by `ravel()`.
 - The **order** in which these are manipulated is an optional parameter.
 - C** : C-style, row-major (default), rightmost (last) dimension changes fastest
 - F** : Fortran-style, column-major, leftmost (first) dimension changes fastest
 - A** : column-major if Fortran-contiguous in memory
 - K** : as elements occur in memory
- Transpose dimensions with `ndarray.T`.
 - For 2D, they are flipped.
 - For >2D, the dimension order is reversed.

Utility:

- Prepare for linear algebra, machine learning, batch processing, I/O, custom looping...
 - ...because some downstream tool insists on a different structure!

```
In [ ]: # Example of reshape: 1D to 3D
np.arange(1, 73).reshape(6, 4, 3)
```

```
In [ ]: # Flatten array2 (one row at a time)
array2.flatten()
```

Element-wise Operations

The standard arithmetic operators like `+`, `-`, `*`, and `/` will act element-wise on arrays and can be **broadcast** across a whole array.

Operations with many functions including `np.round`, `np.abs`, `np.log`, `np.exp`, `np.power`, `np.sin`, `np.cos`, etc. can be mapped element-wise simply by applying them to the array.

```
In [ ]: values = np.arange(1, 17).reshape(4, 4)
values
```

```
In [ ]: values / 4
```

```
In [ ]: # round as a function
np.round(np.exp(values), 2)
```

```
In [ ]: # round as a method
np.exp(values).round(2)
```

When operating on **two arrays**, `numpy` compares their shapes element-wise. It starts with **trailing dimension** and works its way left. Two dimensions are **compatible** when

- they are of equal size, or
- one of them is size 1.

If these conditions are not met, a `ValueError: operands could not be broadcast together` exception is thrown, indicating that the arrays have incompatible shape.

```
In [ ]: # Multiply arrays of the same dimension and shape
a = np.arange(1, 10).reshape(3, 3)
b = np.flip(a) ## np.arange(9, 0, -1).reshape(3, 3)
c = a * b
print(a)
print(b)
print(c)
```

```
In [ ]: # Adding a vector to a matrix
# add vector to each row
a = np.arange(1, 21).reshape(5, 4)
b = np.arange(1000, 4001, 1000) # len(b) = 4
c = a + b
c
```

```
In [ ]: # Adding a vector to a matrix (incompatible dimensions)
# add vector to each column
a = np.arange(1, 21).reshape(4, 5)
b = np.arange(1000, 4001, 1000) # len(b) = 4
c = a + b
c
```

```
In [ ]: b[:, np.newaxis].shape
```

```
In [ ]: # Adding a vector to a matrix (incompatible dimensions, realigned)
a = np.arange(1, 21).reshape(4, 5)
b = np.arange(1000, 4001, 1000) # len(b) = 4
c = a + b[:, np.newaxis]
c
```

Further functions exist for linear algebra operations, e.g.,:

- `np.dot()`
- `np.matmul()`

```
In [ ]: # Dot product (same length)
a = np.array([4, 3, 2, 1])
b = np.array([2, 4, 1, 3])
np.dot(a, b)
```

```
In [ ]: # Matrix multiplication can be used on arrays with appropriate dimensions

c = np.arange(1, 13).reshape(4, 3)
```

```
d = np.arange(13, 1, -1).reshape(3, 4)
np.matmul(c, d)
```

Basic Statistics

`min()`

- Returns minimum

`argmin()`

- Returns indices of minimum along an axis

`max()`

- Returns maximum

`argmax()`

- Returns indices of maximum along an axis

`mean()`

- Returns the arithmetic mean

`std()`

- Returns the standard deviation

```
In [ ]: # Create array for calculation
test = np.random.randint(1, 10, (4, 5))
test
```

```
In [ ]: # Minimum
test.min()
```

```
In [ ]: # Index of minimum (in flattened array)
test.argmax()
```

```
In [ ]: # See the flattened array
test.flatten()
```

```
In [ ]: # **FIRST Index** of minimum, *along* axis 1 (returns shape of axis 0)
test.argmax(axis=1)
```

```
In [ ]: # Average all of the array values
test.mean()
```

```
In [ ]: # Compute the average in each column (across each row)
test.mean(axis=0)
```

```
In [ ]: # Compute the standard deviation of the array values
test.std()
```

Data Selection

- **Comparison operators** can be used to process array expressions in place of `if-elif-else` statements.

- `np.where` acts as a vectorised version of the **ternary** expression.

x **if** condition **else** y

- `np.nonzero` is often used to determine the multidimensional index locations meeting a condition.

```
In [ ]: # Recall values
array2
```

```
In [ ]: # See where values exceed a threshold
array2 > 2
```

```
In [ ]: # Return values meeting a condition
array2[array2 > 2]
```

```
In [ ]: # Return values in column 1 corresponding to the indices
# in column 0 that exceed the mean of column 0
array2[:, 1][array2[:, 0] > array2[:, 0].mean()]
```

```
In [ ]: # Modify values meeting a condition
array2[array2 < 0] **= 2
array2
```

```
In [ ]: # Simple application of where function with scalars
np.where(array2 > 1000, 'large', 'small')
```

```
In [ ]: # Simple application of np.where() function with arrays
a = np.zeros((5, 5)) + 600
b = np.zeros((5, 5)) - 1
c = np.arange(1, 26).reshape(5, 5, order='F')
np.where(c % 3 == 0, a, b)
```

```
In [ ]: # As above, different 'a'
a = np.arange(1, 26).reshape(5, 5, order='F')**2
b = np.zeros((5, 5)) - 1
c = np.arange(1, 26).reshape(5, 5, order='F')
np.where(c % 3 == 0, a, b)
```

```
In [ ]: # Obtain the index locations for elements meeting a condition
# Returns a tuple of arrays
locations = np.nonzero(c % 3 == 0)
locations
```

```
In [ ]: # Index with the resulting tuple or arrays
c[locations]

# Equivalently,
# c[locations[0], locations[1]]
```

Connecting Numpy to Pandas

- Most of these operations are available to be applied directly to `pandas` data structures.
- `pandas` data `.values` can be accessed as `numpy` arrays for further manipulation.

```
In [ ]: import pandas as pd
diamonds = pd.read_csv('./data/diamonds.csv')
diamonds.head()
```

```
In [ ]: diamonds[['x', 'y', 'z']].values
```

Final Points

Third-party utilities like Pandas, NumPy, Matplotlib, Seaborn, and scikit-learn let you move data around quickly, but understanding what you are doing, why you are doing it, and what assumptions you are imposing will always matter more than the code itself.

1. Tools don't replace thinking.
 - Speed and convenience can encourage carelessness.
2. Know what your operations actually mean.
 - `groupby`, `resample`, `rolling` make assumptions about the data and analysis intent.
 - If you cannot explain the line of code, then you are not yet in control of it.
3. Data cleaning is not neutral.
 - Every action changes the story (and sometimes the interesting bits are in the messy spaces!).
4. Be suspicious of defaults.
 - They just promise functionality, not usefulness.
5. Remember to consider scale, uncertainty, and physical meaning.
 - Pandas will happily average quantities that should not be averaged, correlate nonsense with nonsense, and merge incomparable datasets.
 - Your domain knowledge is the only safeguard against confident, but meaningless, results.

Summary

We explored the **Data Science Process**:

- Frame the Problem
- Collect Data
- Exploratory Data Analysis
 - Identifying types of data, missing data, outliers, relationships, structure
- Data Pre-Processing
 - Cleaning, transforming, reducing, engineering
- In-Depth Analysis
- Communicate Results

`pandas` is a Python library for **manipulating** and **analysing** data.

- It is included in Anaconda3 by default.
- Data structures for vector (`Series`) and tabular (`DataFrame`) data
- Methods to enable Data Science practices
- Easy reading and writing
- Simple statistics of the data
- Re-arranging tables to enable discovery

- Grouping data to summarise multiple data points in different categories.

NumPy is the backbone for many Python libraries.

- The main data structure in `numpy` is the `ndarray`.
- Arrays can be vectors and matrices.
- Many efficient array operations can be performed with ease.

Further Reading:

- Han, J. (2023). Data Mining: Concepts and Techniques, 3rd Edition. Elsevier Science & Technology.
 - **Chapters 1 & 2**
 - McKinney, Wes (2022). Python for Data Analysis : Data Wrangling with Pandas, NumPy, and IPython, O'Reilly Media, Incorporated.
 - **Chapters 4, 5, 6, 7, 8, and 10**
 - Mukhiya S. K., and U. Ahmed (2020). Hands-On Exploratory Data Analysis with Python. Packt Publishing.
 - **ALL Chapters**
 - VanderPlas, Jake (2022). Python Data Science Handbook. O'Reilly Media, Incorporated.
 - **Parts 2 & 3**
-

In This Week's Practical

- **Data Manipulation and Preprocessing**
 - **Philadelphia Bike Share Live Data**
 - **Iris Flower Data**
 - **Working with NumPy**
-

Next Session (Following the Mid-Semester Break Week...)

Data Visualisation